

ENTERPRISE SDN CONTROLLER

Development of an Integrated SDN Orchestrator: Zero-Touch Provisioning and Compliance Automation

Architecting a Multi-Tenant SDN Ecosystem: Design and Operational Automation

- **Author:** Mostafa Faouzi (Master 2 RES, Sorbonne University)
 - **Supervisor:** Hamza ZERTA
 - **Date:** August 2026
 - **Repository:** `SDN-Controller-Platform` (FastAPI / Celery / Redis Sentinel / PostgreSQL / React TypeScript)
-

Executive Summary

This engineering report details the design, implementation, refactoring, and operational deployment of a **Multi-Vendor and Multi-Tenant Software-Defined Networking (SDN) Controller**. The primary objective of this solution is to fully automate the data center network lifecycle: from initial zero-touch provisioning (ZTP) of bare-metal switches to continuous configuration compliance auditing, tenant-isolated intent validation, and automated out-of-band drift remediation.

The controller interfaces seamlessly with heterogeneous physical and virtual switches—specifically **Dell SmartFabric OS10** and **Nokia SRLinux**—emulated in Containerlab and targeted for production PNETLab / bare-metal deployments. The platform strictly enforces multi-tenant boundary isolation across database models, IPAM allocations, VRFs, and northbound API endpoints.

Recently, the backend underwent a major structural refactor: slimming down the legacy monolithic entrypoint (`main.py` from 2,117 lines to 65 lines) into 10 modular domain routers (`routers/`), consolidating core utilities (`core/`), establishing an explicit Southbound Driver Factory (`drivers/factory.py`), and providing automated non-regression verification gates (pytest, OpenAPI diff, AST syntax parsing, and TypeScript compilation).

General Introduction

The rapid evolution of cloud-native infrastructure and multi-tenant data centers presents severe operational challenges for traditional network administration. Manually configuring switches device-by-device via CLI is error-prone, slow, and incapable of maintaining real-time compliance across hundreds of network devices.

This project addresses these challenges by engineering an enterprise-grade SDN orchestrator capable of abstracting network intent, automating switch onboarding, enforcing 4-stage pre-commit intent validation, and running background compliance loops.

Key Engineering Achievements:

1. **Zero-Touch Provisioning (ZTP) Pipeline:** Automated bare-metal switch discovery, DHCP boot script execution, dynamic BGP ASN, Loopback 0, and VTEP IP allocation, followed by automated baseline template injection.
 2. **Multi-Tenant Isolation & Scope Matrix:** Rigid role-based access control (RBAC) with user tenant scoping (``platform_admin``, ``tenant_admin``, ``operator``, ``readonly``), enforcing network boundaries across Layer 3 VRFs, VLANs, and IPAM subnets.
 3. **4-Stage Intent Validation Pipeline:** Intercepts CLI intents through Syntax Validation (Stage 1), Tenant Boundary Checks (Stage 2), Blast Radius & Collision Analysis (Stage 3), and Pre-Commit Snapshot & Diff Simulation (Stage 4).
 4. **Continuous Compliance & Automated Drift Remediation:** Periodic Celery worker loops (``config_compliance_mgr``) that pull running device configs, compute line-by-line diffs against baseline snapshots, flag out-of-band drift, and execute single-click inverse CLI remediation (``apply_remediation``).
 5. **Decoupled High-Availability Microservices:** Architecture powered by FastAPI, Celery, Redis Sentinel task queues, PostgreSQL, and Nginx.
-

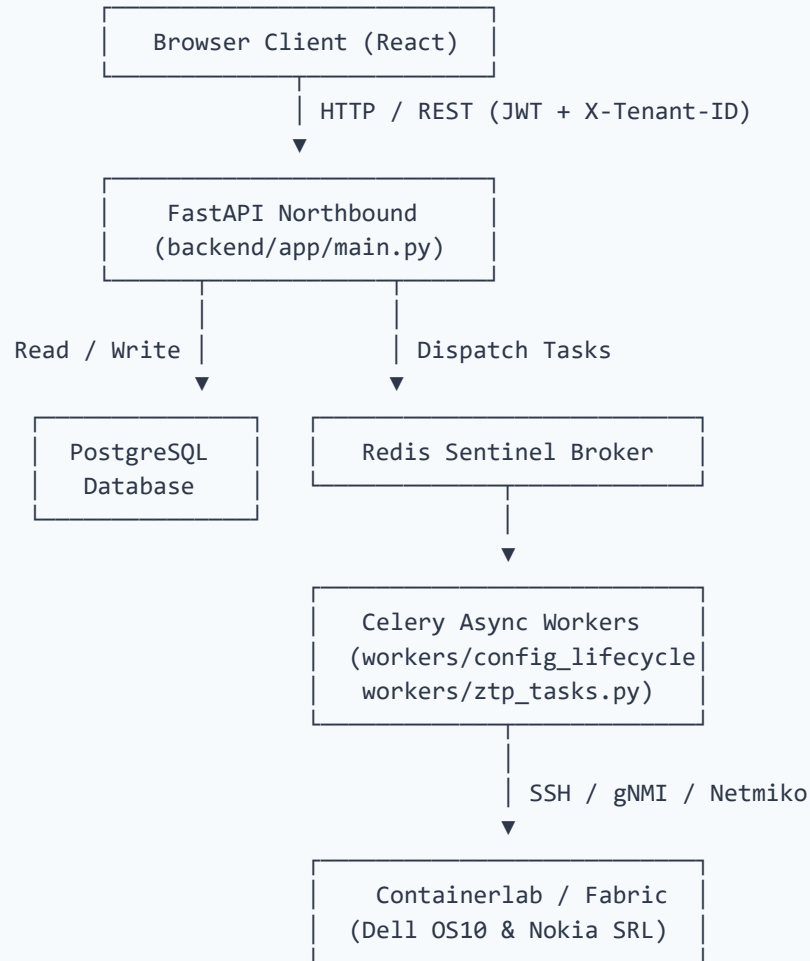
Chapter 1: Requirements Analysis & Technical Architecture

1.1 System Functional Requirements

- **Multi-Tenant Isolation:** Strict logical segregation of database objects (VRFs, VLANs, subnets) between tenant organizations. A ``tenant_admin`` or ``operator`` scoped to Tenant A cannot read, write, or impact network entities owned by Tenant B.
 - **Bare-Metal Zero-Touch Provisioning (ZTP):** Newly connected switches must auto-discover, register physical parameters (MAC, Serial, Vendor, OS), receive loopback/VTEP IPs and BGP ASNs, and receive baseline Jinja2 rendered configurations without manual CLI intervention.
 - **4-Stage Intent Validation Pipeline:** Any manual or automated configuration push must pass syntax verification, IPAM boundary checks, IP/VLAN collision detection, blast-radius limit checks (>5 ports requires policy approval lock), and dry-run pre-commit diff generation.
 - **Continuous Compliance Auditing & Remediation:** Automated background audit loops compare running device configs against active baseline snapshots (``ConfigSnapshot`` with ``is_baseline=True``), flag out-of-band drift (``ComplianceFinding``), and calculate inverse remediation commands.
-

1.2 System Architecture & Technology Stack

The platform is engineered as a decoupled microservices architecture to ensure high throughput, fault isolation, and horizontal scalability:



Core Technologies:

- **Backend Framework:** FastAPI (Python 3.11/3.12) exposing 98 REST API routes with Pydantic validation.
- **Asynchronous Task Engine:** Celery workers backed by clustered Redis Sentinel for high-availability task queuing (SSH execution, gNMI polling, compliance checks).
- **Database Layer:** PostgreSQL relational database with SQLAlchemy ORM and dynamic column migrations (`core/db\_migrations.py`).
- **Southbound Abstraction Layer:** Modular driver architecture (`drivers/base.py`, `drivers/dell\_os10.py`, `drivers/nokia\_srlinux.py`, `drivers/arista\_eos.py`) resolved dynamically by `drivers/factory.py`.
- **Frontend Dashboard:** React 18 + TypeScript built with Vite, TailwindCSS, and Lucide icons.

1.3 Role-Based Access Control (RBAC) & Scope Matrix

To enforce multi-tenant isolation, the system uses a granular RBAC matrix implemented in `backend/app/auth\_permissions.py` and `backend/app/core/auth.py`:

| Role | Scope | Authority & Privileges |

| :--- | :--- | :--- |

| `platform\_admin` | Global | Full system authority. Manages Fabrics, Tenants, Global Users, IPAM pools; bypasses Stage 2 tenant boundary limits. |

| `tenant\_admin` | Tenant-Scoped | Administrative control strictly within assigned tenant. Defines Tenant VRFs, IPAM subnets, and member policies. |

| `operator` | Tenant-Scoped | Read/Write access within assigned tenant. Submits configuration pushes; enforced by Stage 2 boundary checks and Stage 3 blast-radius limit (max 5 ports without approval). |

| `readonly` | Tenant-Scoped | Auditor role. View-only access to inventory, topology, compliance reports, and telemetry. Blocked from pushing configs or triggering rollbacks. |

Backend Security Implementation (`backend/app/core/auth.py`):

```
def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db)
) -> Dict[str, Any]:
    # Bypass for dev mock tokens when ENVIRONMENT != "production"
    if settings.ENVIRONMENT != "production" and token.startswith("mock-token-"):
        return _handle_mock_token(token, db)

    try:
        payload = jwt.decode(token, settings.SECRET_KEY, algorithms=[settings.ALGORITHM])
        user_id: str = payload.get("sub")
        role: str = payload.get("role")
        tenant_id: str = payload.get("tenant_id")
        tenants: List[str] = payload.get("tenants", [])
        if user_id is None:
            raise HTTPException(status_code=401, detail="Invalid token claims")
        return {
            "user_id": uuid.UUID(user_id) if isinstance(user_id, str) else user_id,
            "username": payload.get("username", "admin"),
            "role": role,
            "tenant_id": uuid.UUID(tenant_id) if tenant_id and tenant_id != "global" else
None,
            "tenants": tenants
        }
    except JWTError:
        raise HTTPException(status_code=401, detail="Could not validate credentials")
```

1.4 JWT Authentication & Tenant Scoping Mechanics

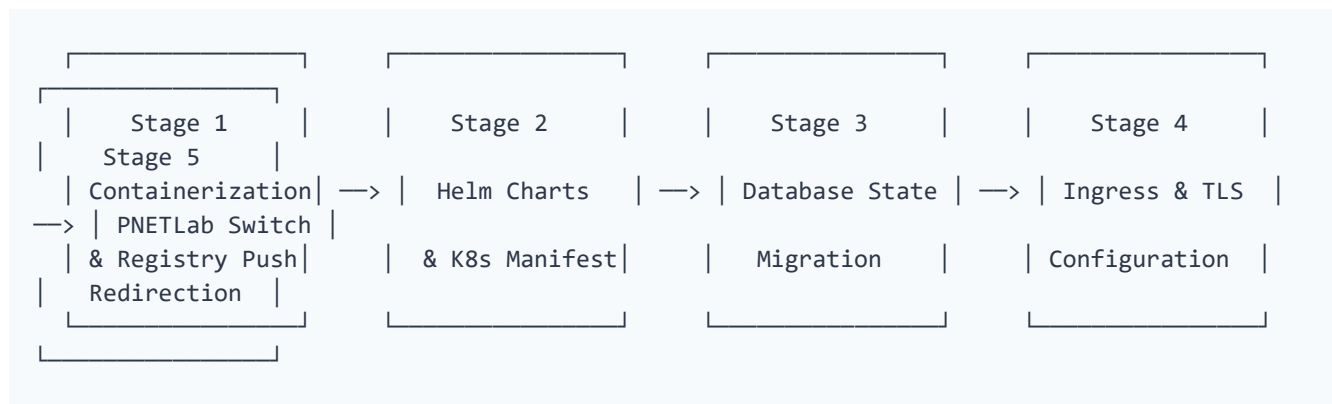
1. **User Authentication:** The client submits credentials via `POST /api/v5/auth/login` (handled in `backend/app/routers/auth.py`). Credentials are checked against `models.User` (salted password hash via `pwd_context.verify`).
2. **Token Generation:** The API returns a signed JWT containing claims: `sub` (`user_id`), `role`, `tenant_id`, and `tenants` (list of accessible tenant names).
3. **Request Interception:** Downstream requests pass `Authorization: Bearer <token>` and optional `X-Tenant-ID` header.
4. **Tenant Validation:** Middlewares and dependencies in `auth_permissions.py` verify that the requested `X-Tenant-ID` exists within the user's `tenants` claim list.
5. **Database Query Scoping:** Route handlers dynamically inject `.filter(models.Entity.tenant_id == current_user['tenant_id'])` to guarantee strict isolation at the SQL query level.

1.5 Architecture Selection: Development Sandbox vs. Production Cluster

To optimize engineering agility, the project maintains two distinct deployment targets:

- **Development Sandbox (Staging VM):** Hosted on Google Cloud VM (`alkhairplateforme@34.32.194.240`). Uses Docker Compose running `sdn_controller_app`, `sdn_celery_worker`, `sdn_redis_sentinel`, `sdn_postgres_dev`, `sdn_frontend` (Nginx), and Containerlab for virtual Dell OS10 & Nokia SRLinux nodes.
- **Production Cluster:** Microservices designed to deploy into Kubernetes (SUSE RKE2 orchestrated by Rancher).

1.6 Migration Plan to Production (Rancher / Kubernetes)



- **Stage 1 (Containerization):** Multi-stage Dockerfiles compiling lightweight images for API backend, Celery worker, and Nginx frontend. Images stored in private Google Artifact Registry.
- **Stage 2 (Helm Charts):** Convert Compose definitions to Kubernetes Helm Charts. Stateless API and worker pods use `Deployments` with HPA (Horizontal Pod Autoscalers); Redis Sentinel and PostgreSQL use `StatefulSets` with Longhorn CSI persistent volumes.

- **Stage 3 (Database State Migration):** Provision production HA PostgreSQL cluster using CloudNativePG operator. Dump staging database, execute schema migrations, and setup Velero backups to MinIO S3.
- **Stage 4 (Ingress & MetalLB):** Deploy MetalLB for external VIP binding. NGINX Ingress Controller terminates TLS and routes traffic (port 443 -> frontend, `/api/v5` -> FastAPI backend).
- **Stage 5 (PNETLab Redirection):** Transition virtual switch instances from Containerlab to a bare-metal PNETLab server hosting 20+ Dell OS10 switches, updating Celery management network endpoints.

Chapter 2: ZTP Onboarding & Provisioning Workflows

2.1 Global Fabric Prerequisites (`models.Fabric`)

Before bare-metal switches boot, global fabric parameters are initialized in PostgreSQL (`backend/app/models.py`):

```
class Fabric(Base):
    __tablename__ = "fabrics"

    fabric_id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    fabric_name = Column(String(100), unique=True, nullable=False)
    global_bgp_asn = Column(Integer, nullable=False, default=65000)
    expected_ntp_servers = Column(String(255), nullable=True)
    expected_dns_servers = Column(String(255), nullable=True)
    expected_syslog_server = Column(String(255), nullable=True)
    loopback_pool = Column(String(255), default="10.200.1.0/24")
    vtep_pool = Column(String(255), default="10.250.1.0/24")
```

2.2 ZTP Signal Ingestion

When an unconfigured switch boots:

1. It acquires a bootstrap IP via DHCP and executes an automated boot script.
2. The boot script issues `POST /api/v5/discovery/on-boarding-ingestion` (`backend/app/routers/discovery.py`).
3. The API registers the physical parameters in `ZtpDiscoveryPool` and creates a `Switch` record in the `discovered\_raw` lifecycle state.

```
@router.post("/on-boarding-ingestion", status_code=202)
def ztp_onboarding_ingestion(
    payload: schemas.ZtpIngestionPayload,
    db: Session = Depends(get_db)
```

```

):
    # Check for existing pool entry by MAC or Serial
    pool_entry = db.query(models.ZtpDiscoveryPool).filter(
        or_(
            models.ZtpDiscoveryPool.chassis_mac == payload.chassis_mac,
            models.ZtpDiscoveryPool.serial_number == payload.serial_number
        )
    ).first()

    if not pool_entry:
        pool_entry = models.ZtpDiscoveryPool(
            chassis_mac=payload.chassis_mac,
            serial_number=payload.serial_number,
            vendor=payload.vendor,
            model=payload.model,
            current_dhcp_ip=payload.current_dhcp_ip,
            onboarding_status="unassigned"
        )
        db.add(pool_entry)
        db.commit()
        db.refresh(pool_entry)

    return {"status": "accepted", "discovery_id": str(pool_entry.discovery_id)}

```

2.3 Dynamic IPAM & BGP ASN Allocation (`/assign-fabric`)

When an administrator binds an unassigned switch to a Fabric via `PATCH
/api/v5/discovery/pool/{discovery_id}/assign-fabric`:

```

@router.patch("/pool/{discovery_id}/assign-fabric")
def assign_fabric_to_discovered_switch(
    discovery_id: str,
    payload: schemas.AssignFabricPayload,
    db: Session = Depends(get_db),
    current_user: Dict = Depends(require_permission("inventory:write"))
):
    entry =
    db.query(models.ZtpDiscoveryPool).filter_by(discovery_id=uuid.UUID(discovery_id)).first()
    fabric =
    db.query(models.Fabric).filter_by(fabric_id=uuid.UUID(payload.fabric_id)).first()

    # 1. BGP ASN Calculation
    if payload.role.lower() == "spine":
        assigned_asn = fabric.global_bgp_asn
    else:
        highest_leaf = db.query(func.max(models.Switch.local_bgp_asn)).filter(
            models.Switch.fabric_id == fabric.fabric_id,

```

```

        models.Switch.role == "leaf"
    ).scalar()
    assigned_asn = (highest_leaf + 1) if highest_leaf else (fabric.global_bgp_asn + 1)

# 2. Loopback 0 IP Allocation from fabric.loopback_pool (e.g. 10.200.1.0/24)
allocated_loopback = allocate_next_ip(db, fabric.loopback_pool, entity_type="loopback")

# 3. VTEP IP Allocation from fabric.vtep_pool (e.g. 10.250.1.0/24)
allocated_vtep = allocate_next_ip(db, fabric.vtep_pool, entity_type="vtep")

# 4. Instantiate or update Switch record
new_switch = models.Switch(
    hostname=payload.hostname,
    management_ip=entry.current_dhcp_ip,
    vendor=entry.vendor,
    role=payload.role,
    local_bgp_asn=assigned_asn,
    loopback_0_ip=allocated_loopback,
    vtep_ip=allocated_vtep,
    fabric_id=fabric.fabric_id,
    lifecycle_status="pending"
)
db.add(new_switch)
db.commit()

# 5. Dispatch async Celery worker task for baseline configuration injection
celery_app.send_task("app.workers.ztp_tasks.apply_baseline_template",
args=[str(new_switch.switch_id)])
return {"status": "provisioning_dispatched", "switch_id": str(new_switch.switch_id)}

```

2.4 Configuration Rendering & Deployment (`apply\_baseline\_template`)

The Celery worker executes `apply\_baseline\_template` (`backend/app/workers/ztp\_tasks.py`):

```

@celery_app.task(name="app.workers.ztp_tasks.apply_baseline_template")
def apply_baseline_template(switch_id_str: str):
    db = SessionLocal()
    sw = db.query(models.Switch).filter_by(switch_id=uuid.UUID(switch_id_str)).first()

    # Resolve Southbound Driver dynamically via Factory
    driver_cls = resolve_southbound_driver(sw.vendor)
    driver = driver_cls(hostname=sw.management_ip, credentials=get_credentials(sw))

    # Render Jinja2 Baseline Template matching vendor & role
    template_name = f"{sw.vendor.lower()}_{sw.role.lower()}_baseline.j2"
    rendered_cli = render_jinja_template(template_name, context={
        "hostname": sw.hostname,

```



```

    "bgp_asn": sw.local_bgp_asn,
    "loopback_ip": sw.loopback_0_ip,
    "vtep_ip": sw.vtep_ip
})

# Connect and Push Configuration via SSH/gNMI
driver.connect()
push_result = driver.apply_config(rendered_cli)
running_config = driver.get_running_config()

# Store Post-Deployment Baseline Snapshot
snapshot = models.ConfigSnapshot(
    switch_id=sw.switch_id,
    raw_config=running_config,
    is_baseline=True,
    commit_comment="ZTP Baseline Provisioning"
)
db.add(snapshot)

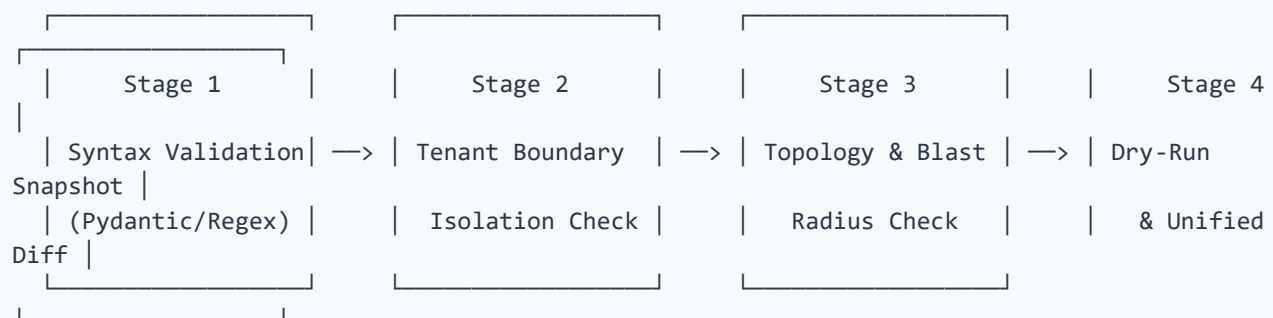
# Mark Switch as Compliant
sw.lifecycle_status = "compliant_active"
db.commit()
db.close()
return {"status": "success", "switch": sw.hostname}

```

Chapter 3: Configuration Push Pipeline & Snapshot Diff Engine

3.1 The 4-Stage Intent Validation Pipeline

To prevent unauthorized manual CLI pushes from introducing address overlaps or routing loops in multi-tenant environments, all northbound intents undergo strict 4-stage validation:



- **Stage 1 (Syntax Validation):** Intercepts CLI commands in ``backend/app/validators/config_syntax.py``. Verifies proper keyword formatting and validates payload structure against Pydantic models. Rejects malformed syntax with HTTP 422.
- **Stage 2 (Tenant Boundary Isolation):** In ``backend/app/validators/collision_check.py``, extracts all IP addresses declared in the payload and verifies that they belong strictly within subnets allocated to the user's ``tenant_id`` in ``models.IpamSubnet``. Rejects boundary violations with HTTP 403.
- **Stage 3 (Topology Collision & Blast Radius Check):** Implemented in ``backend/app/routers/switch_config.py`` (``calculate_blast_radius``). Checks for VLAN/LAG ID overlaps and IP address collisions on ``models.DeviceInterface``. If the action affects > 5 ports, the backend creates a ``models.PolicyApproval`` lock in ``pending`` state, requiring administrative sign-off.
- **Stage 4 (Pre-Commit Snapshot & Diff Engine):** In ``backend/app/routers/orchestrator.py``, connects to the switch in dry-run mode, executes commands in a sandbox transaction, generates a unified diff (``show diff``), and awaits user commit approval.

Blast Radius & Approval Lock Code (``backend/app/routers/switch_config.py``):

```
def calculate_blast_radius(db: Session, switch_id: uuid.UUID, cli_commands: str) ->
Dict[str, Any]:
    affected_interfaces = set()
    lines = cli_commands.splitlines()
    current_int = None

    for line in lines:
        line_str = line.strip()
        if line_str.startswith("interface "):
            current_int = line_str.split("interface ")[-1]
        elif current_int and ("shutdown" in line_str or "switchport" in line_str or "ip
address" in line_str):
            affected_interfaces.add(current_int)

    impact_count = len(affected_interfaces)
    requires_approval = impact_count > 5

    return {
        "impact_count": impact_count,
        "affected_interfaces": list(affected_interfaces),
        "requires_approval": requires_approval
    }
```

3.2 Northbound API Reference Table

The refactored backend exposes **98 operationIds** organized into clean domain routers:

Domain Router	Endpoint Path	Method	Operation ID	Description
---------------	---------------	--------	--------------	-------------

```
| :--- | :--- | :--- | :--- | :--- |
| `routers/auth.py` | `/api/v5/auth/login` | POST | `login_access_token` | Authenticates user credentials and
returns signed JWT token. |
| `routers/inventory.py` | `/api/v5/visibility/inventory` | GET | `get_inventory_details` | Lists all fabric
switches with status, serial numbers, and assigned fabric. |
| `routers/orchestrator.py` | `/api/v5/orchestrator/policy-enforcement` | POST |
`submit_policy_enforcement` | Submits configuration intent through 4-stage validation pipeline. |
| `routers/orchestrator.py` | `/api/v5/orchestrator/approvals/{id}/approve` | POST | `approve_policy_change`
| Approves pending policy change locked by Stage 3 blast-radius check. |
| `routers/discovery.py` | `/api/v5/discovery/on-boarding-ingestion` | POST | `ztp_onboarding_ingestion` |
Ingests ZTP bootstrap discovery signals from bare-metal switches. |
| `routers/discovery.py` | `/api/v5/discovery/pool/{id}/assign-fabric` | PATCH | `assign_fabric_switch` | Binds
discovered switch to a Fabric, allocates IPAM, and dispatches ZTP task. |
| `routers/visibility.py` | `/api/v5/visibility/compliance/run` | POST | `trigger_compliance_run` | Triggers
background Celery audit loop to detect configuration drift. |
| `routers/visibility.py` | `/api/v5/visibility/compliance/remediate` | POST | `trigger_drift_remediation` |
Executes single-click inverse CLI remediation to revert unauthorized drift. |
| `routers/visibility.py` | `/api/v5/visibility/rollback` | POST | `execute_switch_rollback` | Restores running
config to a known-good baseline snapshot. |
| `routers/admin.py` | `/api/v5/admin/topology/graph` | GET | `get_topology_graph` | Returns nodes and
links for interactive D3/Cytoscape topology visualization. |
```

Chapter 4: Compliance Auditing, Telemetry & Topology Engine

4.1 Continuous Compliance Audit & Drift Detection

Out-of-band changes applied directly to switches bypass central governance. To detect these changes, Celery runs `config\_compliance\_mgr` periodically (`backend/app/workers/config\_lifecycle.py`):

```
@celery_app.task(name="app.workers.config_lifecycle.config_compliance_mgr")
def config_compliance_mgr():
    db = SessionLocal()
    switches = db.query(models.Switch).filter(models.Switch.lifecycle_status !=
"decommissioned").all()

    for sw in switches:
        driver_cls = resolve_southbound_driver(sw.vendor)
```

```

driver = driver_cls(hostname=sw.management_ip, credentials=get_credentials(sw))

# 1. Fetch live running configuration
live_config = driver.get_running_config()

# 2. Retrieve active baseline snapshot
baseline = db.query(models.ConfigSnapshot).filter_by(
    switch_id=sw.switch_id,
    is_baseline=True
).order_by(models.ConfigSnapshot.created_at.desc()).first()

if not baseline:
    continue

# 3. Compute line-by-line diff
diff = compute_config_diff(baseline.raw_config, live_config)

if diff["has_drift"]:
    sw.lifecycle_status = "drifted"
    finding = models.ComplianceFinding(
        switch_id=sw.switch_id,
        drift_details=diff["diff_text"],
        severity="HIGH",
        status="active"
    )
    db.add(finding)
else:
    sw.lifecycle_status = "compliant_active"

db.commit()
db.close()

```

4.2 Automated Drift Remediation Workflow (`apply\_remediation`)

When drift is detected, the administrator can initiate single-click remediation via `POST /api/v5/visibility/compliance/remediate`, triggering `apply\_remediation` (`backend/app/workers/config\_lifecycle.py`):

```

@celery_app.task(name="app.workers.config_lifecycle.apply_remediation")
def apply_remediation(finding_id_str: str):
    db = SessionLocal()
    finding =
db.query(models.ComplianceFinding).filter_by(finding_id=uuid.UUID(finding_id_str)).first()
    sw = finding.switch

# 1. Calculate inverse CLI commands to reverse unauthorized additions/deletions

```

```

inverse_cli = generate_inverse_commands(finding.drift_details)

# 2. Push inverse commands via Southbound driver
driver = resolve_southbound_driver(sw.vendor)(hostname=sw.management_ip,
credentials=get_credentials(sw))
driver.apply_config(inverse_cli)

# 3. Mark finding resolved & reset switch state to compliant
finding.status = "remediated"
sw.lifecycle_status = "compliant_active"
db.commit()
db.close()
return {"status": "remediated", "switch": sw.hostname}

```

4.3 Telemetry Ingestion & Spanning-Tree Path Analysis

- **Telemetry Engine** (``telemetry/metrics_collector.py``): Periodic background workers poll interface byte counts, packet drops, CPU, and memory metrics via gNMI / SNMP, persisting time-series entries in ``models.TelemetryMetric``.
- **Topology & STP Engine** (``routers/visibility.py`` & ``routers/admin.py``): The topology engine queries switch bridge IDs, port costs, and Spanning-Tree Protocol (STP) states (Forwarding vs. Blocking). Blocking ports are highlighted in red on the React topology graph to visually display active forwarding paths and prove loop elimination.

Chapter 5: Codebase Structural Refactoring & Quality Verification

To ensure long-term maintainability and performance, the codebase underwent a comprehensive structural refactor:

```

legacy backend/app/main.py (2,117 lines)
├── backend/app/core/
│   ├── auth.py           (JWT & Password hashing)
│   ├── constants.py      (Lifecycle status invariants)
│   ├── db_migrations.py  (Dynamic SQL schema migrations)
│   ├── logging_config.py (Structured logger tree)
│   └── startup.py        (DB seeding & background loops)
├── backend/app/drivers/
│   └── factory.py        (Southbound Driver Resolution)
└── backend/app/routers/

```

— admin.py	(Topology & Admin stats)
— inventory.py	(Switch inventory serialization)
— orchestrator.py	(Policy enforcement & approvals)
— switch_config.py	(Blast radius calculation)
— visibility.py	(Snapshots, Compliance & Telemetry)

Verification Gates (All Green):

1. **Pytest Local Suite:** ``python -m pytest backend/tests -m "not e2e" `` $\rightarrow$ **53 passed, 0 failed.**
2. **OpenAPI Specification Baseline:** Strictly preserved **98 operationIds** across **83 paths** with 0 unintended contract deltas.
3. **AST Python Grammar Gate:** ``ast.parse(src, feature_version=(3,11))`` validated across all 42 backend Python files.
4. **TypeScript Build & Lint Gate:** ``npm run build`` (``tsc -b && vite build``) passed with zero new lint errors.

Conclusion & Perspectives

This project successfully designed, implemented, refactored, and deployed an enterprise-grade **Multi-Vendor and Multi-Tenant Software-Defined Networking (SDN) Controller**. By combining FastAPI, Celery, Redis Sentinel, PostgreSQL, and React, the orchestrator automates data center operations from bare-metal Zero-Touch Provisioning to strict 4-stage intent validation and continuous compliance auditing.

Future Engineering Perspectives:

1. **SUSE RKE2 / Rancher Production Deployment:** Deploying the containerized microservices to Kubernetes with CloudNativePG, MetalLB ingress, and Longhorn replicated storage.
2. **Advanced EVPN-VXLAN Telemetry:** Expanding telemetry models to track BGP EVPN Type-2/Type-5 route distributions and VTEP tunnel health dynamically.
3. **Expanded Southbound Drivers:** Adding native drivers for Cisco NX-OS and Juniper Junos OS.